

Encrypted Password Manager

Project Documentation — Digital Security

Nenu Mihai-Adrian

Digital Security

July 6, 2026

Abstract

This report documents a local password manager that stores credentials encrypted and grants access through a single master password. The application uses AES-256 in GCM mode for authenticated encryption, the memory-hard Scrypt function for deriving the encryption key from the master password, and an SQLite database that holds only ciphertext. Both a command-line and a graphical interface are provided over a shared cryptographic backend. The document describes the design and its justification, the development steps, the problems encountered and how they were solved, the automated tests, and an honest analysis of the threat model and remaining limitations.

Contents

1	Overview and relevance	2
2	How the requirements are satisfied	2
3	Architecture	2
4	Cryptographic design and justification	3
4.1	Deriving the key (Scrypt)	3
4.2	Encrypting the data (AES-256-GCM)	3
4.3	Verifying the master password	3
4.4	What the database contains	4
5	Installation and running	4
6	User interface	4
7	Development steps	7
8	Problems encountered and solutions applied	8
9	Testing	8
10	Critical analysis, threat model and limitations	8
11	Conclusion	9

1 Overview and relevance

Passwords are still the primary authentication mechanism online, and the reuse of weak passwords is one of the most common causes of account compromise. A password manager solves this by letting the user remember a single strong secret — the *master password* — while every account receives a long, unique, randomly generated password.

This project implements such a manager as a small, self-contained application and demonstrates several core topics of digital security in practice: symmetric encryption (AES-256-GCM), key derivation from a human password (Scrypt), encryption at rest, integrity and tamper detection through authenticated encryption, and cryptographically secure random generation.

2 How the requirements are satisfied

The brief required AES-256, one of Argon2 / scrypt / PBKDF2, and encrypted SQLite. Table 1 maps each requirement to its implementation.

Requirement	Implementation
AES-256	AES-256-GCM (AESGCM from the <code>cryptography</code> library), 256-bit key.
Argon2 / scrypt / PBKDF2	Scrypt , a memory-hard KDF, with $N = 2^{15}$, $r = 8$, $p = 1$.
Encrypted SQLite	Every credential is AES-GCM encrypted <i>before</i> being written; the file holds only opaque blobs and non-secret KDF parameters.
Master-password access	The key is derived from the master password at runtime; a stored <i>verifier</i> confirms correctness without storing the password.

Table 1: Mapping of brief requirements to the implementation.

Note on Argon2 vs Scrypt. The brief allows any of the three KDFs. Argon2id is the modern first choice, but it requires a third-party package that was unavailable in the build environment. Scrypt is also memory-hard, ships inside the audited `cryptography` library, and fully satisfies the requirement. The code isolates key derivation in a single function, so switching to Argon2id later is a one-line change.

3 Architecture

The application is split into three layers so that cryptography, storage and the user interface remain independent and testable.

File	Responsibility
<code>crypto_utils.py</code>	Script key derivation, AES-256-GCM encrypt/decrypt, secure password generation. No knowledge of storage.
<code>vault.py</code>	Opens the SQLite file, verifies the master password, performs encrypted create/read/update/delete.
<code>pwmanager.py</code>	Command-line interface.
<code>pwmanager_gui.py</code>	Graphical interface (Tkinter) over the same backend.
<code>selftest.py</code>	Automated functional and security tests.

Table 2: Project modules and their responsibilities.

4 Cryptographic design and justification

4.1 Deriving the key (Scrypt)

A human password is far too weak and short to be used directly as an AES key. It is passed through Scrypt, which stretches it into a 32-byte (256-bit) key, mixes in a random 16-byte salt so that identical master passwords yield different keys (defeating precomputed tables), and is memory-hard — roughly 32 MB per attempt with the chosen parameters — making large-scale brute-force guessing extremely costly while a single legitimate unlock stays fast.

```

1 def derive_key(master_password, salt, n=2**15, r=8, p=1):
2     kdf = Scrypt(salt=salt, length=32, n=n, r=r, p=p) # 32 bytes = AES-256
3     return kdf.derive(master_password.encode("utf-8"))

```

Listing 1: Key derivation with Scrypt.

4.2 Encrypting the data (AES-256-GCM)

Every secret is encrypted with AES-256 in GCM mode. GCM is preferred over a plain mode such as CBC because it is *authenticated*: it provides confidentiality and, through its authentication tag, integrity and authenticity. Any modification of the ciphertext is detected at decryption time, which gives tamper detection for free. Two rules are enforced: a fresh random 96-bit nonce is generated for every encryption (nonce reuse in GCM is catastrophic), and associated data binds each ciphertext to its role.

```

1 def encrypt(key, plaintext, associated_data=b''):
2     aes = AESGCM(key)
3     nonce = secrets.token_bytes(12) # unique per encryption
4     return nonce + aes.encrypt(nonce, plaintext, associated_data)

```

Listing 2: Authenticated encryption; a unique nonce per call.

4.3 Verifying the master password

We must be able to report a wrong password without storing the password or key. At vault creation a known constant is encrypted with the derived key and saved as a *verifier*. On unlock, a candidate key is derived and used to decrypt the verifier: success means the password is correct; an authentication failure means it is wrong. The verifier leaks nothing — it is just another ciphertext.

4.4 What the database contains

Because the entire entry (title, username, password, URL, notes, timestamps) is inside the encrypted blob, an attacker who steals the file learns only how many entries exist. This is the sense in which the SQLite storage is “encrypted”: encryption is applied at the application level to every value before it reaches the database.

5 Installation and running

Requirements: Python 3.10+ and the `cryptography` package; the GUI also uses `tkinter`, bundled with standard Python / Anaconda.

```
1 pip install -r requirements.txt
2 python3 pwmanager_gui.py      # graphical interface
3 python3 pwmanager.py          # command-line interface
```

Listing 3: Running the application.

6 User interface

The graphical interface presents an unlock screen, an entry list with live search, and a details panel from which a stored password can be revealed or copied.

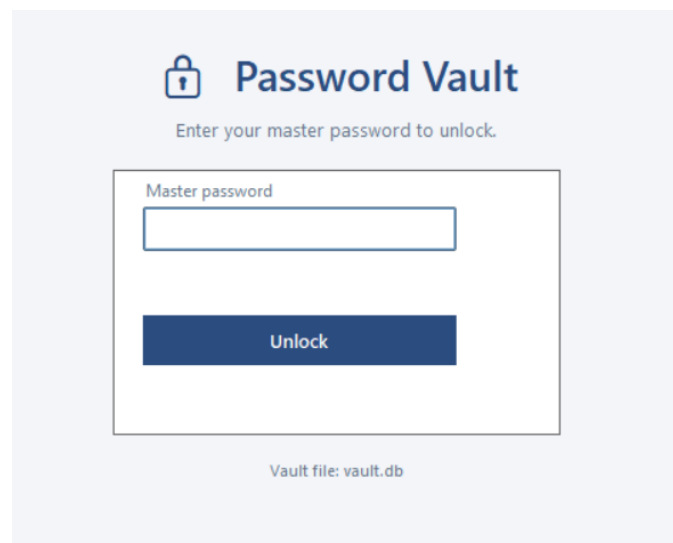


Figure 1: Unlock / create screen. The master password is entered hidden.

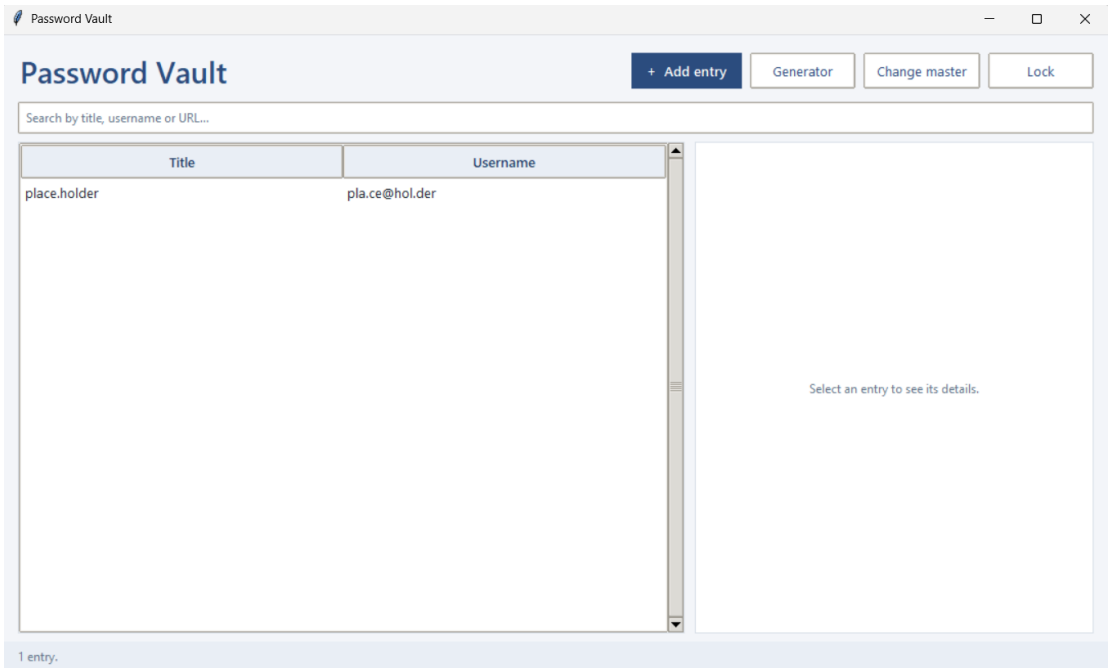


Figure 2: Main window: entry list on the left, details on the right.

place.holder

USERNAME

pla.ce@hol.der

Copy

PASSWORD

[]eQb35\$%vish2a7kbRT

Hide

Copy

URL

asd

NOTES

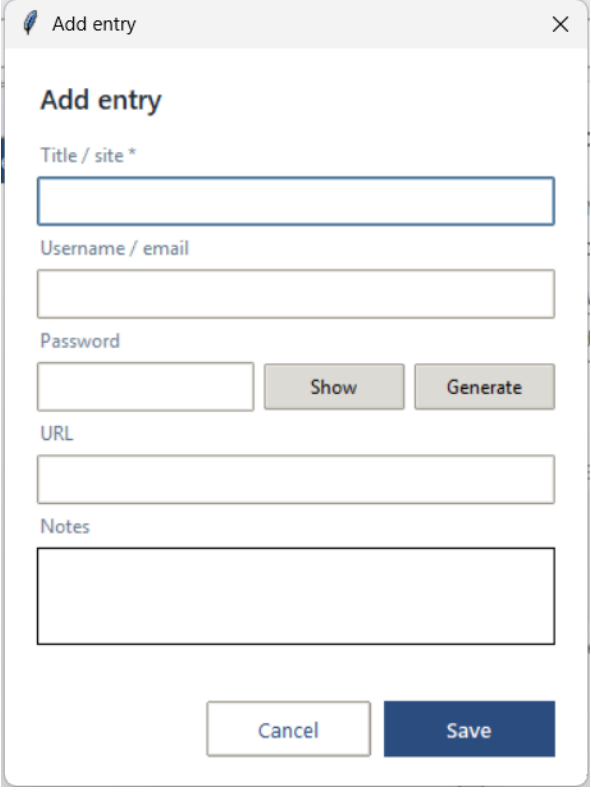
no

Created 2026-07-06 11:15 · Modified 2026-07-06 11:45

Edit

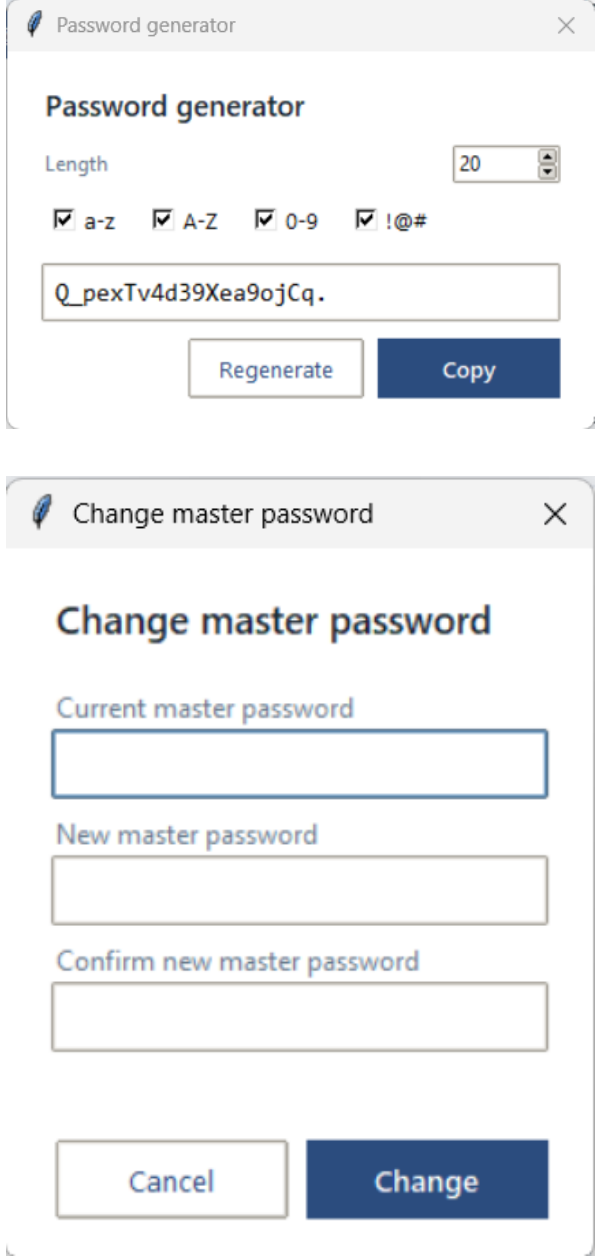
Delete

Figure 3: Viewing a stored password via the Show/Hide toggle, with one-click copy and automatic clipboard clearing.



The 'Add entry' dialog box features a title bar with a feather icon and a close button. The main area is titled 'Add entry' and contains four input fields: 'Title / site *', 'Username / email', 'Password', and 'URL'. The 'Password' field has 'Show' and 'Generate' buttons to its right. Below these is a larger 'Notes' text area. At the bottom are 'Cancel' and 'Save' buttons.

Figure 4: Add / edit dialog with a built-in generator.



The 'Password generator' dialog box has a title bar with a feather icon and a close button. It is titled 'Password generator' and includes a 'Length' spinner set to 20. Below are four checked checkboxes: 'a-z', 'A-Z', '0-9', and '!@#'. A text field displays the generated password 'Q_pexTv4d39Xea9ojCq.'. 'Regenerate' and 'Copy' buttons are at the bottom.

The 'Change master password' dialog box also has a title bar with a feather icon and a close button. It is titled 'Change master password' and contains three input fields: 'Current master password', 'New master password', and 'Confirm new master password'. 'Cancel' and 'Change' buttons are at the bottom.

Figure 5: Password generator and master-password change.

7 Development steps

1. Chose a layered architecture (crypto / storage / UI) so each concern could be tested independently.
2. Implemented `crypto_utils.py`: Script derivation, AES-GCM helpers, and a CSPRNG-based password generator.
3. Built `vault.py`: the two-table schema, initialize/unlock flow, the verifier mechanism, and encrypted CRUD.
4. Added the master-password change routine with full re-encryption inside a single transaction.
5. Wrote the command-line interface, then a polished Tkinter GUI over the same backend.

6. Wrote an automated self-test and iterated until every check passed.

8 Problems encountered and solutions applied

Problem	Solution
Argon2 unavailable (no package, no network).	Used Scrypt, memory-hard and part of the cryptography library; the KDF is isolated so Argon2id can replace it with one line.
Detecting a wrong master password without storing password or key.	A <i>verifier</i> : a known constant encrypted with the key; correct key decrypts it, wrong key fails authentication.
“Encrypted SQLite” without a native full-disk library.	Application-level encryption: every value is AES-GCM encrypted before storage, so the file is only ciphertext.
GCM nonce reuse would break security.	A fresh random 96-bit nonce for every encryption, stored with each ciphertext.
Ciphertext role confusion (swapping blobs).	Associated data binds each ciphertext to its purpose (entry vs. verifier).
Atomicity while changing the master password.	Re-encrypt everything inside a single transaction with rollback on error.
Metadata leaking through the database.	Store the whole entry inside the encrypted blob, so only the entry count is visible.

Table 3: Problems encountered during development and the solutions applied.

9 Testing

The script `selftest.py` runs twelve automated checks, all passing: initialization and unlock; add and retrieve round-trip; sorted listing; search; update; delete; a locked vault refusing access; wrong-password rejection and correct acceptance; **no plaintext of any secret in the raw database file**; master-password change re-encrypting all data; **tamper detection** when a stored blob is modified; and generator output of the requested length containing all character classes.

10 Critical analysis, threat model and limitations

Protected against: theft of the database file (only ciphertext at rest), offline brute-force of the master password (slowed massively by Scrypt), and silent modification of stored data (detected by GCM authentication).

Not protected against (honest limitations): a compromised machine while the vault is unlocked (the key and decrypted secrets live in RAM, and Python cannot reliably wipe them); a weak master password, which no KDF can rescue; and metadata such as the number of entries, which remains visible.

Possible improvements: replace Scrypt with Argon2id; adopt SQLCipher for transparent full-file encryption that also hides metadata; add an auto-lock timeout (already present in the GUI),

a clipboard copy with automatic clearing (also present), and a breach/reuse checker; and provide encrypted export/import for backups.

11 Conclusion

The application meets the brief: it stores credentials encrypted with AES-256-GCM, derives the key from the master password with the memory-hard Scrypt function, and persists only ciphertext in SQLite. Beyond the minimum, it adds tamper detection, a strong password generator, master-password re-keying, auto-lock, and both a command-line and a graphical interface, together with a documented threat model that states clearly what the design does and does not protect against.